LAPORAN PRAKTIKUM MATA KULIAH

METODE NUMERIK

PRAKTIKUM 06 – SPL



DISUSUN OLEH:

L0325021 – DUTA LATHIIF HAQIM

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

2026

BAB I

ANALISIS SOURCE CODE

1. Penyelesaian SPL: Eliminasi Gauss, Gauss-Jordan, Dekomposisi LU, Jacobi, dan Gauss-Seidel

1.Analisis Source Code "jenis *source code*"

```
% ============================================================
% SOAL 1 : Solusi SPL dengan Gauss, Gauss-Jordan, LU,
%          Jacobi, dan Gauss-Seidel (maks iterasi 20, tol 0.001)
% Jalankan:  >> soal1
% ============================================================
1;
function x = gauss_elim(A, b)
  % Eliminasi Gauss + pivoting parsial + substitusi mundur
  n = length(b); Ab = [A b];
  for k = 1:n-1
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    for i = k+1:n
      m = Ab(i,k) / Ab(k,k);
      Ab(i,k:n+1) = Ab(i,k:n+1) - m * Ab(k,k:n+1);
    end
  end
  x = zeros(n,1);
  for i = n:-1:1
    x(i) = (Ab(i,n+1) - Ab(i,i+1:n) * x(i+1:n)) / Ab(i,i);
  end
end

function x = gauss_jordan(A, b)
  % Eliminasi Gauss-Jordan
  n = length(b); Ab = [A b];
  for k = 1:n
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    Ab(k,:) = Ab(k,:) / Ab(k,k);
    for i = [1:k-1, k+1:n]
      Ab(i,:) = Ab(i,:) - Ab(i,k) * Ab(k,:);
    end
  end
  x = Ab(:, n+1);
end

function x = lu_solve(A, b)
  % Dekomposisi LU (Doolittle): A = L*U, Ly = b, Ux = y
  n = length(b); L = eye(n); U = zeros(n);
  for i = 1:n
```

i. Analisis source code

Tujuan program: menyelesaikan SPL 5 persamaan 5 variabel (p, q, r, s, t) dengan tiga metode langsung (Gauss, Gauss-Jordan, LU Doolittle) dan dua metode iteratif (Jacobi, Gauss-Seidel). Hasilnya dibandingkan. Untuk metode iteratif, tebakan awal $x^0 = 0$, toleransi 0.001, dan maksimum 20 iterasi.

Sistem dan data yang dipakai kode:

$$A = \begin{bmatrix} 5 & -1 & -1 & 0 & 0 \\ -1 & 5 & -1 & -1 & 0 \\ -1 & -1 & 4 & -1 & -1 \\ 0 & 0 & 1 & 4 & -2 \\ 0 & 1 & -1 & 1 & 4 \end{bmatrix}$$

$$x = \begin{bmatrix} p \\ q \\ r \\ s \\ t \end{bmatrix}$$

$$b = \begin{bmatrix} -1 \\ 2 \\ 6 \\ 2 \\ -1 \end{bmatrix}$$

Alur kode secara berurutan:

- Fungsi dan data: lima fungsi didefinisikan (diawali 1; agar file dibaca sebagai script), lalu A, b, x0, tol, maxit, dan nama variabel diisi.
- Pemanggilan: kelima metode dipanggil, lalu hasilnya dicetak berdampingan dan dilengkapi tabel iterasi.

Metode Eliminasi Gauss (gauss_elim)

Kode membentuk matriks augmented lalu memilih pivot parsial (nilai |a| terbesar di kolom k, baris k sampai n) dan menukar baris bila perlu. Pengali dan eliminasinya:

m_ik=a_ik/a_kk
baris_i←baris_i−m_ik·baris_k

Setelah menjadi segitiga atas, dilakukan substitusi mundur:

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{ij} x_j}{a_{ii}}$$

Metode Gauss-Jordan (gauss_jordan)

Pivot parsial dipakai lagi. Baris pivot dinormalisasi (dibagi a_kk), lalu elemen kolom k dieliminasi di semua baris lain (di atas dan di bawah pivot), sehingga A menjadi I dan kolom terakhir langsung berisi x. Tidak perlu substitusi mundur.

Metode LU Doolittle (lu_solve)

Diagonal L bernilai 1. Rumus yang dipakai kode:

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}$$

$$l_{ji} = \frac{a_{ji} - \sum_{k=1}^{i-1} l_{jk} u_{ki}}{u_{ii}}$$

Setelah itu dilakukan substitusi maju dan mundur:

$$Ly = b$$
$$Ux = y$$
$$y_i = b_i - \sum_{j=1}^{i-1} l_{ij} y_j$$
$$x_i = \frac{y_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}$$

Metode Jacobi (jacobi)

Semua komponen iterasi baru dihitung dari iterasi lama saja:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}$$

Untuk SPL ini, rumus per variabelnya:

$$p^{(k+1)} = \frac{-1 + q^{(k)} + r^{(k)}}{5}$$
$$q^{(k+1)} = \frac{2 + p^{(k)} + r^{(k)} + s^{(k)}}{5}$$
$$r^{(k+1)} = \frac{6 + p^{(k)} + q^{(k)} + s^{(k)} + t^{(k)}}{4}$$
$$s^{(k+1)} = \frac{2 - r^{(k)} + 2t^{(k)}}{4}$$
$$t^{(k+1)} = \frac{-1 - q^{(k)} + r^{(k)} - s^{(k)}}{4}$$

Metode Gauss-Seidel (gauss_seidel)

Nilai terbaru langsung dipakai dalam iterasi yang sama. Hal ini terlihat dari kode, yang memperbarui x(i) di tempat:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}}{a_{ii}}$$

Kriteria berhenti kedua metode iteratif (selisih maksimum antar iterasi, bukan galat terhadap solusi eksak):

$$\varepsilon_k = \max_i \left| x_i^{(k)} - x_i^{(k-1)} \right| < 0.001$$

Kelemahan dan kondisi khusus:

- Gauss: gagal bila semua kandidat pivot nol (matriks singular). Kode tidak mengecek kondisi ini. Kompleksitasnya $O(n^3)$.
- Gauss-Jordan: kelemahannya sama, dengan operasi sekitar 1.5 kali lebih banyak dari Gauss, dan tetap $O(n^3)$.
- LU Doolittle: pada kode ini tanpa pivoting. Bila u_ii = 0, terjadi pembagian nol, sehingga perlu pivoting (PA = LU) untuk kasus umum. Faktorisasi $O(n^3)$, setiap substitusi $O(n^2)$. Keunggulannya, L dan U bisa dipakai ulang untuk banyak vektor b.
- Jacobi dan Gauss-Seidel: syarat cukup konvergensi adalah matriks dominan diagonal. Cek per baris:

|a_ii|≥∑_(j≠i)▒|a_ij|

Baris 1: 5 > 2, baris 2: 5 > 3, baris 3: 4 = 4 (hanya sama, tidak ketat), baris 4: 4 > 3, baris 5: 4 > 3. Jadi A dominan diagonal lemah dengan ketidaksamaan ketat pada sebagian baris. Matriksnya tidak dapat direduksi (setiap variabel terhubung ke variabel lain), sehingga konvergensi tetap terjamin. Kompleksitasnya $O(n^2)$ per iterasi.

Catatan kode (tidak ada kesalahan fatal, hanya hal kecil):

- Pesan '(Jacobi belum mencapai toleransi dalam 20 iterasi)' bersifat tetap (hard-coded) pada angka 20. Sebaiknya diganti dengan maxit, misalnya fprintf(' (Jacobi belum mencapai toleransi dalam %d iterasi)\n', maxit);.
- Nama variabel hist menimpa fungsi bawaan Octave hist. Tidak error, tetapi lebih aman memakai nama lain seperti riwayat.

2.Analisis “hasil *source code*”

```
================ SOAL 1 : SOLUSI SPL ================
Var     Gauss           Gauss-Jordan    LU              Jacobi          Gauss-Seidel
p       0.32544379      0.32544379      0.32544379      0.32509187      0.32568239
q       0.83234714      0.83234714      0.83234714      0.83204321      0.83258862
r       1.79487179      1.79487179      1.79487179      1.79448861      1.79505801
s       0.04142012      0.04142012      0.04142012      0.04155804      0.04134867
t       -0.01972387     -0.01972387     -0.01972387     -0.01985528     -0.01971982

Jacobi       : 9 iterasi, selisih terakhir = 0.000597
Gauss-Seidel : 5 iterasi, selisih terakhir = 0.000689

Tabel iterasi Jacobi  [iter | p q r s t | selisih maks]:
   1.0000  -0.2000   0.4000   1.5000   0.5000  -0.2500   1.5000
   2.0000   0.1800   0.7600   1.6125        0  -0.1000   0.5000
   3.0000   0.2745   0.7585   1.7100   0.0469  -0.0369   0.0975
   4.0000   0.2937   0.8063   1.7608   0.0541  -0.0238   0.0508
   5.0000   0.3134   0.8217   1.7825   0.0479  -0.0249   0.0218
   6.0000   0.3209   0.8288   1.7895   0.0419  -0.0218   0.0074
   7.0000   0.3237   0.8305   1.7924   0.0417  -0.0203   0.0029
   8.0000   0.3246   0.8316   1.7939   0.0417  -0.0199   0.0014
   9.0000   0.3251   0.8320   1.7945   0.0416  -0.0199   0.0006
Tabel iterasi Gauss-Seidel  [iter | p q r s t | selisih maks]:
   1.0000e+00  -2.0000e-01   3.6000e-01   1.5400e+00   1.1500e-01   1.6250e-02   1.5400e+00
   2.0000e+00   1.8000e-01   7.6700e-01   1.7696e+00   6.5734e-02  -1.5793e-02   4.0700e-01
   3.0000e+00   3.0731e-01   8.2852e-01   1.7964e+00   4.2993e-02  -1.8768e-02   1.2731e-01
   4.0000e+00   3.2499e-01   8.3289e-01   1.7955e+00   4.1735e-02  -1.9774e-02   1.7681e-02
   5.0000e+00   3.2568e-01   8.3259e-01   1.7951e+00   4.1349e-02  -1.9720e-02   6.8922e-04

Himpunan penyelesaian (Gauss):
  {(p,q,r,s,t)} = {(0.325444, 0.832347, 1.794872, 0.041420, -0.019724)}
octave:2>
```

ii. Analisis hasil

Tabel solusi (kolom Var, Gauss, Gauss-Jordan, LU, Jacobi, Gauss-Seidel)

Ketiga metode langsung memberi hasil identik sampai 8 angka desimal. Dua metode iteratif sedikit berbeda karena berhenti saat ε_k < 0.001, bukan saat galat benar-benar nol. Hasil metode langsung dalam bentuk pecahan eksak:

$$p = \frac{165}{507} = 0.32544379$$
$$q = \frac{422}{507} = 0.83234714$$
$$r = \frac{910}{507} = 1.79487179$$
$$s = \frac{21}{507} = 0.04142012$$
$$t = -\frac{10}{507} = -0.01972387$$

Verifikasi dengan substitusi balik (semua dikalikan 507 agar bilangan bulat):

$$5(165) - 422 - 910 = -507 \Rightarrow -\frac{507}{507} = -1$$
$$-165 + 5(422) - 910 - 21 = 1014 \Rightarrow \frac{1014}{507} = 2$$
$$-165 - 422 + 4(910) - 21 - (-10) = 3042 \Rightarrow \frac{3042}{507} = 6$$
$$910 + 4(21) - 2(-10) = 1014 \Rightarrow \frac{1014}{507} = 2$$
$$422 - 910 + 21 + 4(-10) = -507 \Rightarrow -\frac{507}{507} = -1$$

Kelima ruas kanan sama dengan b, sehingga solusi terbukti benar.

Informasi jumlah iterasi dan selisih terakhir

- Jacobi: 9 iterasi, selisih terakhir 0.000597 < 0.001.
- Gauss-Seidel: 5 iterasi, selisih terakhir 0.000689 < 0.001.
- Pesan "belum mencapai toleransi" tidak tampil, artinya kedua metode memenuhi toleransi sebelum iterasi ke-20.

Galat terhadap solusi eksak (pada iterasi terakhir):

$$\max_i \left| x_i^{Jacobi} - x_i \right| \approx 3.83 \times 10^{-4} (pada\ r)$$
$$\max_i \left| x_i^{GS} - x_i \right| \approx 2.42 \times 10^{-4} (pada\ q)$$

Keduanya lebih kecil dari toleransi 0.001.

Tabel iterasi Jacobi dan Gauss-Seidel

Kolom pertama adalah nomor iterasi, kolom 2 sampai 6 adalah nilai p, q, r, s, t, dan kolom terakhir adalah selisih maksimum ε_k. Pada Jacobi iterasi 2, nilai s tampil "0" karena memang tepat nol:

$$s^{(2)} = \frac{2 - 1.5 + 2(-0.25)}{4} = 0$$

Pada tabel Gauss-Seidel, Octave memakai notasi ilmiah (e+00, e−01) karena rentang nilai kolom sangat lebar. Itu hanya soal tampilan.

Cek iterasi pertama Jacobi ($x^0 = 0$):

$$p^{(1)} = -\frac{1}{5} = -0.2$$
$$q^{(1)} = \frac{2}{5} = 0.4$$
$$r^{(1)} = \frac{6}{4} = 1.5$$
$$s^{(1)} = \frac{2}{4} = 0.5$$
$$t^{(1)} = -\frac{1}{4} = -0.25$$

Selisih maksimumnya 1.5 (pada r), sesuai kolom terakhir.

Cek iterasi pertama Gauss-Seidel (nilai terbaru langsung dipakai):

$$p^{(1)} = -\frac{1}{5} = -0.2$$
$$q^{(1)} = \frac{2 + (-0.2) + 0 + 0}{5} = 0.36$$
$$r^{(1)} = \frac{6 + (-0.2) + 0.36 + 0 + 0}{4} = 1.54$$
$$s^{(1)} = \frac{2 - 1.54 + 2(0)}{4} = 0.115$$
$$t^{(1)} = \frac{-1 - 0.36 + 1.54 - 0.115}{4} = 0.01625$$

Hasilnya cocok dengan baris 1 tabel (−0.2, 0.36, 1.54, 0.115, 0.01625) dan selisih 1.54.

Determinan dan faktor LU

Pada eliminasi Gauss untuk matriks ini, pivot parsial tidak memerlukan pertukaran baris. Pengali pertamanya:

$$m_{21} = -\frac{1}{5} = -0.2$$
$$m_{31} = -\frac{1}{5} = -0.2$$

Pivot-pivotnya (perhitungan manual saya, tidak dicetak program):

$$5,4.8,3.5,4.357143,4.155738$$

Determinan:

$$\det(A) = 5 \times 4.8 \times 3.5 \times 4.357143 \times 4.155738 = 1521$$

Karena det(A) ≠ 0, SPL punya solusi tunggal. Nilai 1521 = 39^2 konsisten dengan penyebut solusi 507 (1521 = 3 × 507). Tanpa pertukaran baris, faktor LU Doolittle sama dengan hasil eliminasi Gauss:

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -0.2 & 1 & 0 & 0 & 0 \\ -0.2 & -0.25 & 1 & 0 & 0 \\ 0 & 0 & 0.285714 & 1 & 0 \\ 0 & 0.208333 & -0.214286 & 0.215847 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 5 & -1 & -1 & 0 & 0 \\ 0 & 4.8 & -1.2 & -1 & 0 \\ 0 & 0 & 3.5 & -1.25 & -1 \\ 0 & 0 & 0 & 4.357143 & -1.714286 \\ 0 & 0 & 0 & 0 & 4.155738 \end{bmatrix}$$

Cek sebagian L·U (baris 5 dikalikan kolom 2, 3, 4, 5 dari U):

$$0.208333 \times 4.8 = 1 = a_{52}$$
$$0.208333 \times (-1.2) + (-0.214286)(3.5) = -1 = a_{53}$$
$$0.208333 \times (-1) + (-0.214286)(-1.25) + 0.215847 \times 4.357143 = 1 = a_{54}$$
$$(-0.214286)(-1) + 0.215847 \times (-1.714286) + 4.155738 = 4 = a_{55}$$

Semuanya sama dengan baris 5 matriks A, jadi L·U = A.

Hasil akhir (Gauss, 6 desimal, sama dengan keluaran program):

$$\{(p, q, r, s, t)\} = \{(0.325444,0.832347,1.794872,0.041420, -0.019724)\}$$

Perbandingan antar metode:

- Gauss, Gauss-Jordan, dan LU menghasilkan solusi identik (galat hanya galat pembulatan mesin), karena ketiganya metode langsung dengan $O(n^3)$.
- Gauss-Seidel lebih cepat (5 iterasi) dibanding Jacobi (9 iterasi) dan lebih akurat pada toleransi yang sama, karena memakai nilai terbaru dalam iterasi yang sama.
- Pada SPL kecil seperti ini, metode langsung lebih baik karena hasilnya eksak. Metode iteratif lebih menguntungkan untuk matriks besar dan jarang (sparse).

2. Analisis Galat Solusi SPL terhadap Nilai Eksak (Gauss-Jordan, LU, Jacobi, Gauss-Seidel)

i. Analisis source code

```
% ============================================================
% SOAL 2 : Galat = abs(nilai eksak - nilai hampiran)
%          Nilai eksak diambil dari metode langsung (Gauss)
% Jalankan:  >> soal2
% ============================================================
1;
function x = gauss_elim(A, b)
  % Eliminasi Gauss + pivoting parsial + substitusi mundur
  n = length(b); Ab = [A b];
  for k = 1:n-1
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    for i = k+1:n
      m = Ab(i,k) / Ab(k,k);
      Ab(i,k:n+1) = Ab(i,k:n+1) - m * Ab(k,k:n+1);
    end
  end
  x = zeros(n,1);
  for i = n:-1:1
    x(i) = (Ab(i,n+1) - Ab(i,i+1:n) * x(i+1:n)) / Ab(i,i);
  end
end

function x = gauss_jordan(A, b)
  % Eliminasi Gauss-Jordan
  n = length(b); Ab = [A b];
  for k = 1:n
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    Ab(k,:) = Ab(k,:) / Ab(k,k);
    for i = [1:k-1, k+1:n]
      Ab(i,:) = Ab(i,:) - Ab(i,k) * Ab(k,:);
    end
  end
  x = Ab(:, n+1);
end

function x = lu_solve(A, b)
  % Dekomposisi LU (Doolittle): A = L*U, Ly = b, Ux = y
  n = length(b); L = eye(n); U = zeros(n);
  for i = 1:n
    for j = i:n
```

Tujuan program: menghitung galat mutlak solusi SPL yang sama seperti nomor 1 (matriks A, vektor b, $x^0 = 0$, toleransi 0.001, maksimum 20 iterasi). Hasil Gauss-Jordan, LU, Jacobi, dan Gauss-Seidel dibandingkan dengan nilai eksak. Lima fungsi solver identik dengan nomor 1, jadi bagian yang baru hanya blok setelah pemanggilan solver.

Alur kode secara berurutan:

- Input dan inisialisasi: A, b, x0, tol, maxit, dan nama variabel diisi seperti nomor 1.
- Perhitungan solusi: kelima metode dipanggil. Untuk Jacobi dan Gauss-Seidel hanya dua keluaran yang diambil (x dan it), sehingga tabel iterasi tidak dipakai.
- Penentuan nilai eksak: x_eksak = x_gauss.
- Perhitungan galat: abs(x_eksak - x_metode) untuk empat metode.
- Verifikasi (tampilan): galat tiap variabel dan galat maksimum dicetak dengan format %e, ditambah jumlah iterasi.

Rumus galat mutlak per variabel:

$$E_i = \left| x_i^{eksak} - x_i^{hampiran} \right|$$

Galat maksimum tiap metode:

$$E_{maks} = \max_i E_i$$

Rumus iterasi Jacobi dan Gauss-Seidel serta kriteria berhenti sama seperti nomor 1:

$$\varepsilon_k = \max_i \left| x_i^{(k)} - x_i^{(k-1)} \right| < 0.001$$

Kelemahan dan kondisi khusus:

- Nilai "eksak" di sini sebenarnya hasil numerik Gauss (komentar kode menyebutnya "analitik"). Nilai ini dapat dipakai sebagai pembanding karena galat Gauss hanya galat pembulatan mesin, sekitar 10^(−16). Itulah sebabnya kolom Gauss tidak ada: galatnya nol menurut definisi.
- Galat Gauss-Jordan dan LU mencerminkan galat pembulatan floating point (epsilon mesin Octave ≈ 2.22×10^(−16)), bukan galat metode.
- Galat Jacobi dan Gauss-Seidel adalah galat pemotongan akibat iterasi dihentikan saat ε_k < 0.001. Galat ini bergantung pada toleransi dan tebakan awal.
- Kriteria berhenti memakai selisih antar iterasi, sedangkan tabel ini memakai selisih terhadap solusi eksak. Keduanya berbeda, dan galat sebenarnya biasanya lebih kecil dari selisih terakhir.

Catatan kode: tidak ada kesalahan penulisan. Pada nomor 1 saya menulis galat maksimum Gauss-Seidel ≈ 2.42×10^(−4). Nilai yang lebih tepat dari output ini adalah 2.4148×10^(−4) ≈ 2.41×10^(−4), jadi mohon pakai angka ini di laporan.

ii. Analisis hasil

```
octave:2> soal2

================ SOAL 2 : GALAT |eksak - hampiran| ================
Nilai eksak (Gauss): 0.32544379  0.83234714  1.79487179  0.04142012  -0.01972387

Var    Gauss-Jordan   LU             Jacobi         Gauss-Seidel
p      5.5511e-17     0.0000e+00     3.5192e-04     2.3860e-04
q      1.1102e-16     0.0000e+00     3.0393e-04     2.4148e-04
r      0.0000e+00     0.0000e+00     3.8318e-04     1.8621e-04
s      2.7756e-17     0.0000e+00     1.3792e-04     7.1448e-05
t      5.8981e-17     6.9389e-18     1.3142e-04     4.0468e-06
maks   1.1102e-16     6.9389e-18     3.8318e-04     2.4148e-04

Iterasi dipakai -> Jacobi: 9, Gauss-Seidel: 5
octave:3>
```

Nilai eksak (baris pertama output) sama dengan solusi Gauss nomor 1. Dalam bentuk pecahan, nilainya sama:

$$p = \frac{165}{507} = 0.32544379$$
$$q = \frac{422}{507} = 0.83234714$$
$$r = \frac{910}{507} = 1.79487179$$
$$s = \frac{21}{507} = 0.04142012$$
$$t = -\frac{10}{507} = -0.01972387$$

Tabel galat per variabel (kolom Gauss-Jordan, LU, Jacobi, Gauss-Seidel):

- Gauss-Jordan: galat antara 0 sampai 1.1102×10^(−16), dengan maksimum 1.1102×10^(−16) pada q. Besarnya setara epsilon mesin, sehingga hanya galat pembulatan.
- LU: galat 0 pada p, q, r, s dan 6.9389×10^(−18) pada t. Faktorisasi LU praktis menghasilkan solusi yang sama dengan Gauss.
- Jacobi: galat 10^(−4) pada semua variabel, dengan maksimum 3.8318×10^(−4) pada r.
- Gauss-Seidel: galat lebih kecil, dengan maksimum 2.4148×10^(−4) pada q.

Verifikasi manual beberapa galat dari output nomor 1:

$$E_p^{Jacobi} = |0.32544379 - 0.32509187| = 0.00035192 = 3.5192 \times 10^{-4}$$
$$E_r^{Jacobi} = |1.79487179 - 1.79448861| = 0.00038318 = 3.8318 \times 10^{-4}$$
$$E_q^{GS} = |0.83234714 - 0.83258862| = 0.00024148 = 2.4148 \times 10^{-4}$$
$$E_t^{GS} = |-0.01972387 - (-0.01971982)| = 0.00000405 = 4.05 \times 10^{-6}$$

Semuanya cocok dengan tabel galat. Galat maksimum Jacobi (3.8318×10^(−4)) pada iterasi ke-9 dan Gauss-Seidel (2.4148×10^(−4)) pada iterasi ke-5 sama-sama lebih kecil dari toleransi:

$$3.8318 \times 10^{-4} < 0.001$$
$$2.4148 \times 10^{-4} < 0.001$$

Iterasi yang dipakai: Jacobi 9 iterasi dan Gauss-Seidel 5 iterasi, sama dengan nomor 1.

Perbandingan antar metode dari data galat:

- Urutan akurasi dari yang terbaik: LU (≈ 10^(−18)), Gauss-Jordan (≈ 10^(−16)), Gauss-Seidel (≈ 10^(−4)), Jacobi (≈ 10^(−4)).
- Rasio galat maksimum Jacobi terhadap Gauss-Seidel: 3.8318×10^(−4) / 2.4148×10^(−4) ≈ 1.59. Gauss-Seidel lebih akurat dan memakai iterasi lebih sedikit (5 berbanding 9), karena nilai terbaru langsung dipakai dalam iterasi yang sama.
- Selisih galat antara metode langsung (≈ 10^(−16)) dan iteratif (≈ 10^(−4)) sekitar 12 orde besaran. Galat metode iteratif bisa diperkecil dengan menurunkan toleransi, sedangkan galat metode langsung dibatasi presisi mesin.

- Galat sebenarnya lebih kecil dari selisih terakhir antar iterasi untuk Jacobi ($3.83\times10^{-4} < 5.97\times10^{-4}$) dan Gauss-Seidel ($2.41\times10^{-4} < 6.89\times10^{-4}$). Jadi kriteria berhenti ε_k bersifat konservatif dalam kasus ini.

3.Perbandingan Running Time Lima Metode (tic-toc)

i. Analisis source code

```
% ============================================================
% SOAL 3 : Running time kelima metode dengan tic-toc
%          (hanya algoritma, tanpa input/output)
% Jalankan:  >> soal3
% ============================================================
1;
function x = gauss_elim(A, b)
  % Eliminasi Gauss + pivoting parsial + substitusi mundur
  n = length(b); Ab = [A b];
  for k = 1:n-1
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    for i = k+1:n
      m = Ab(i,k) / Ab(k,k);
      Ab(i,k:n+1) = Ab(i,k:n+1) - m * Ab(k,k:n+1);
    end
  end
  x = zeros(n,1);
  for i = n:-1:1
    x(i) = (Ab(i,n+1) - Ab(i,i+1:n) * x(i+1:n)) / Ab(i,i);
  end
end

function x = gauss_jordan(A, b)
  % Eliminasi Gauss-Jordan
  n = length(b); Ab = [A b];
  for k = 1:n
    [~, idx] = max(abs(Ab(k:n, k))); idx = idx + k - 1;
    if idx ~= k, Ab([k idx], :) = Ab([idx k], :); end
    Ab(k,:) = Ab(k,:) / Ab(k,k);
    for i = [1:k-1, k+1:n]
      Ab(i,:) = Ab(i,:) - Ab(i,k) * Ab(k,:);
    end
  end
  x = Ab(:, n+1);
end

function x = lu_solve(A, b)
  % Dekomposisi LU (Doolittle): A = L*U, Ly = b, Ux = y
  n = length(b); L = eye(n); U = zeros(n);
  for i = 1:n
```

Tujuan program: mengukur waktu komputasi (running time) kelima metode untuk SPL yang sama seperti nomor 1 dan 2. Pengukuran hanya mencakup algoritma solver, tanpa input/output. Karena SPL berukuran kecil (5×5), setiap metode dijalankan N = 1000 kali, lalu waktunya dirata-rata.

Alur kode secara berurutan:

- Input dan inisialisasi: A, b, x0, tol, maxit, dan nama variabel sama seperti nomor 1. Ditambah N = 1000 sebagai jumlah pengulangan.
- Pengukuran: untuk tiap metode, tic memulai stopwatch, perulangan memanggil solver sebanyak N kali, toc membaca total waktu, lalu dibagi N.
- Tampilan: waktu rata-rata tiap metode dicetak dengan format %e, lalu diurutkan dengan sort dan dicetak sebagai urutan tercepat ke terlambat.

Rumus waktu rata-rata per eksekusi:

$$t_{rata} = \frac{T_{total}}{N}$$
$$t_{rata} = \frac{T_{total}}{1000}$$

Kompleksitas teoretis tiap metode (n = jumlah variabel):

$$Gauss: O(n^3)$$
$$Gauss - Jordan: O(n^3)$$
$$LU\ Doolittle: O(n^3)$$
$$Jacobi\ dan\ Gauss - Seidel: O(n^2) per\ iterasi$$

Kompleksitas total metode iteratif bergantung pada jumlah iterasi k:

$$O(k \cdot n^2)$$

Pada nomor 1, k = 9 untuk Jacobi dan k = 5 untuk Gauss-Seidel.

Kelemahan dan kondisi khusus:

- Hasil tic-toc bergantung pada komputer, beban CPU, dan interpreter Octave. Angka tidak persis sama setiap dijalankan, sehingga yang bermakna adalah urutan dan perbandingan relatifnya.
- Octave adalah bahasa interpreter. Waktu banyak dipengaruhi oleh overhead pemanggilan fungsi dan perulangan, bukan hanya jumlah operasi aritmetika. Untuk n = 5, overhead ini mendominasi sehingga urutan waktu tidak selalu mengikuti kompleksitas $O(n^3)$ versus $O(n^2)$.
- Fungsi jacobi dan gauss_seidel menyimpan riwayat iterasi (hist) pada setiap iterasi dan membentuk indeks [1:i-1, i+1:n] di dalam perulangan. Itu menambah beban waktu di luar rumus iterasi intinya.
- Pemanggilan gauss_elim(A, b); diakhiri titik koma sehingga hasilnya tidak ditampilkan, dan tidak ada keluaran yang mengganggu pengukuran. Tidak ada kesalahan penulisan pada kode. Satu catatan kecil: variabel hist menimpa fungsi bawaan hist, seperti pada nomor 1.
- Variabel nama dan tol dideklarasikan tetapi nama tidak dipakai di program ini. Tidak berpengaruh pada hasil.

ii. Analisis hasil

```
================ SOAL 3 : RUNNING TIME (rata-rata 1000x) ================
Gauss            : 3.480558e-04 detik
Gauss-Jordan     : 4.034321e-04 detik
Dekomposisi LU   : 5.064001e-04 detik
Jacobi           : 1.052192e-03 detik
Gauss-Seidel     : 5.919528e-04 detik

Urutan tercepat -> terlambat: Gauss < Gauss-Jordan < LU < Gauss-Seidel < Jacobi
octave:4>
```

Arti setiap output (urut sesuai tampilan), waktu rata-rata per satu kali penyelesaian:

$$t_Gauss = 3.480558 \times 10\text{^}(-4)\ detik$$
$$t_GJ = 4.034321 \times 10\text{^}(-4)\ detik$$
$$t_LU = 5.064001 \times 10\text{^}(-4)\ detik$$
$$t_Jacobi = 1.052192 \times 10\text{^}(-3)\ detik$$
$$t_GS = 5.919528 \times 10\text{^}(-4)\ detik$$

Dalam satuan milidetik (supaya mudah dibandingkan): Gauss 0.348 ms, Gauss-Jordan 0.403 ms, LU 0.506 ms, Gauss-Seidel 0.592 ms, Jacobi 1.052 ms.

Verifikasi urutan: dari nilai di atas, urutan dari kecil ke besar adalah

$$3.4806 \times 10^{-4} < 4.0343 \times 10^{-4} < 5.0640 \times 10^{-4} < 5.9195 \times 10^{-4} < 1.0522 \times 10^{-3}$$

yaitu Gauss < Gauss-Jordan < LU < Gauss-Seidel < Jacobi, sesuai baris terakhir output.

Perbandingan relatif terhadap Gauss (hitungan manual):

$$\frac{t_{GJ}}{t_{Gauss}} = \frac{4.0343}{3.4806} \approx 1.16$$
$$\frac{t_{LU}}{t_{Gauss}} = \frac{5.0640}{3.4806} \approx 1.45$$
$$\frac{t_{GS}}{t_{Gauss}} = \frac{5.9195}{3.4806} \approx 1.70$$
$$\frac{t_{Jacobi}}{t_{Gauss}} = \frac{10.5219}{3.4806} \approx 3.02$$

Rasio Jacobi terhadap Gauss-Seidel:

$$\frac{t_{Jacobi}}{t_{GS}} = \frac{10.5219}{5.9195} \approx 1.78$$

Interpretasi:

- Gauss tercepat. Eliminasi hanya mengerjakan bagian segitiga bawah, lalu substitusi mundur. Operasinya paling sedikit di antara metode langsung.
- Gauss-Jordan sekitar 1.16 kali lebih lambat dari Gauss karena mengeliminasi elemen di atas dan di bawah pivot serta menormalisasi baris pivot. Operasinya kira-kira 1.5 kali Gauss untuk n besar. Pada n = 5 selisihnya lebih kecil karena overhead interpreter.
- LU sekitar 1.45 kali lebih lambat dari Gauss karena memakai perulangan bersarang berindeks (dua perulangan untuk faktorisasi ditambah substitusi maju dan mundur). Overhead perulangan di Octave cukup besar. Keunggulan LU baru terasa bila banyak vektor b yang diselesaikan dengan A yang sama, karena faktorisasi hanya dilakukan sekali.
- Gauss-Seidel lebih cepat dari Jacobi, sejalan dengan jumlah iterasi pada nomor 1 (5 berbanding 9). Perbandingan jumlah iterasinya:

$$\frac{9}{5} = 1.8$$

Rasio waktu 1.78 sangat dekat dengan rasio iterasi 1.8, yang menunjukkan biaya per iterasi kedua metode hampir sama dan perbedaan waktu terutama berasal dari jumlah iterasi.

- Waktu per iterasi (hitungan manual dari output):

$$\frac{t_{Jacobi}}{9} = 1.0522 \times \frac{10^{-3}}{9} \approx 1.169 \times 10^{-4} detik$$
$$\frac{t_{GS}}{5} = 5.9195 \times \frac{10^{-4}}{5} \approx 1.184 \times 10^{-4} detik$$

Keduanya hampir sama, sekitar $1.17 \times 10\text{^}(-4)$ detik per iterasi. Ini mendukung penjelasan di atas.

- Dua metode iteratif lebih lambat dari ketiga metode langsung pada SPL kecil ini. Hal ini berbeda dengan teori untuk matriks besar dan jarang, di mana $O(k \cdot n^2)$ bisa lebih murah daripada $O(n^3)$. Pada n = 5, jumlah operasi sangat kecil sehingga overhead perulangan, pembuatan indeks, dan penyimpanan hist lebih menentukan.

Hasil akhir: urutan waktu dari tercepat ke terlambat adalah

Gauss<Gauss-Jordan<LU<Gauss-Seidel<Jacobi

Perbandingan dengan nomor 1 dan 2:

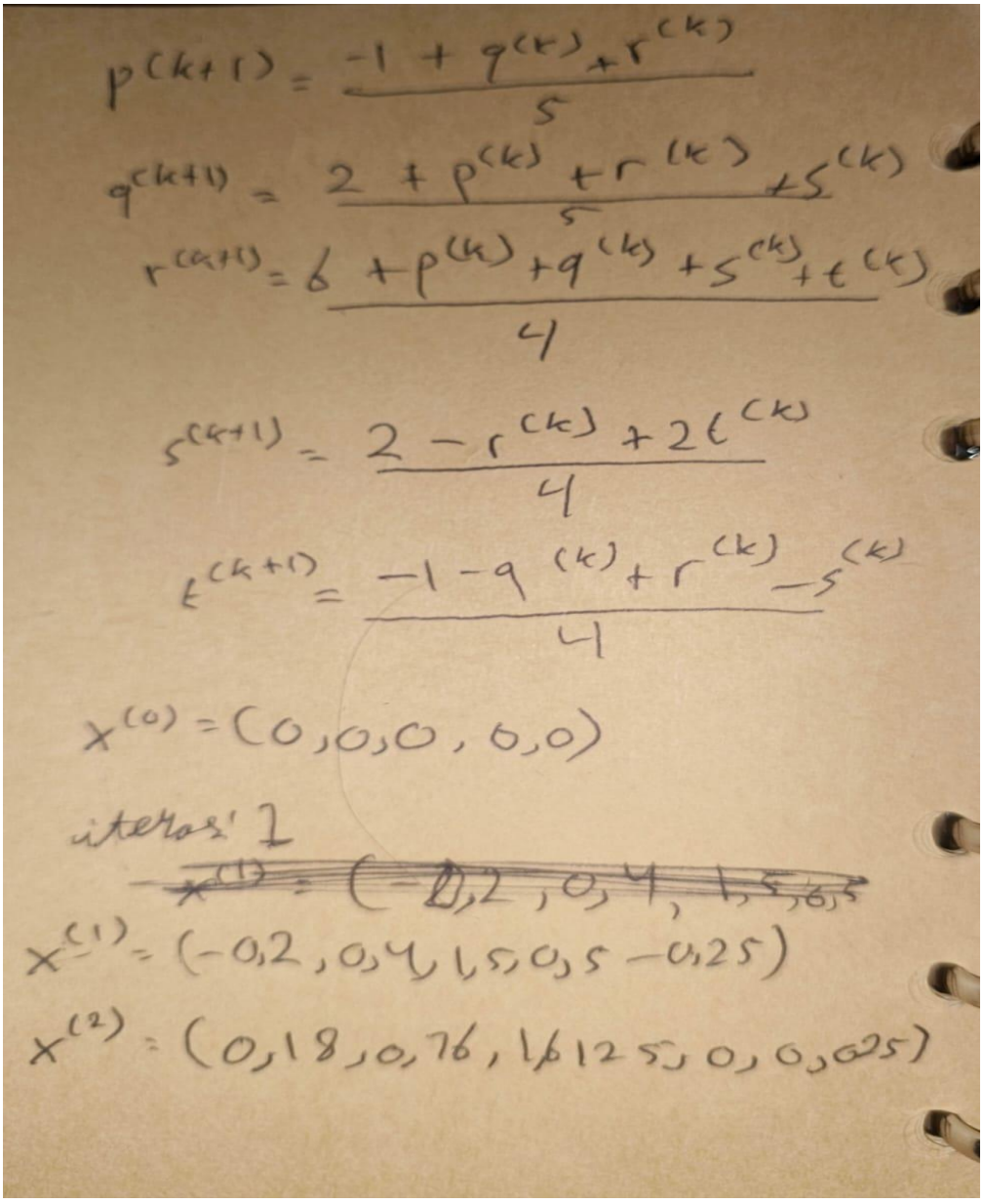
BAB II

ANALISIS NUMERIK

1.Perhitungan Manual Jacobi dan Gauss-Seidel (Tanpa Program)

Sebelum dipakai di laporan, ada tiga ketidakcocokan antara hasil manual Anda dan program (nomor 1) yang perlu diperbaiki. Saya tidak mengubah angka Anda secara diam-diam. Detailnya ada di bagian ii.

Tujuan: menyelesaikan SPL yang sama seperti nomor 1 dengan tangan, memakai tebakan awal nol dan toleransi 0.001.

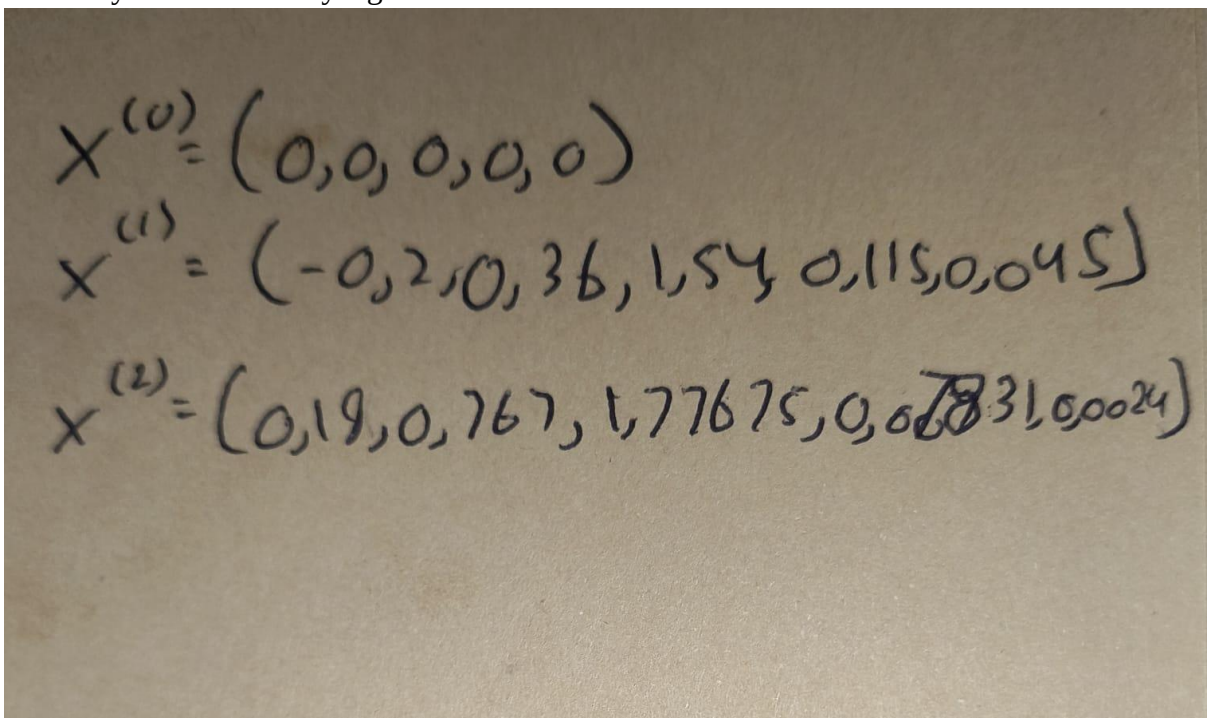


$$x^{(0)} = (0,0,0,0,0)$$

Rumus iterasi Jacobi (semua nilai ruas kanan dari iterasi k):

$$p^{(k+1)} = \frac{-1 + q^{(k)} + r^{(k)}}{5}$$
$$q^{(k+1)} = \frac{2 + p^{(k)} + r^{(k)} + s^{(k)}}{5}$$
$$r^{(k+1)} = \frac{6 + p^{(k)} + q^{(k)} + s^{(k)} + t^{(k)}}{4}$$
$$s^{(k+1)} = \frac{2 - r^{(k)} + 2t^{(k)}}{4}$$
$$t^{(k+1)} = \frac{-1 - q^{(k)} + r^{(k)} - s^{(k)}}{4}$$

2.Rumus Gauss-Seidel sama, tetapi nilai yang baru dihitung langsung dipakai pada baris berikutnya dalam iterasi yang sama:



$$p^{(k+1)} = \frac{-1 + q^{(k)} + r^{(k)}}{5}$$
$$q^{(k+1)} = \frac{2 + p^{(k+1)} + r^{(k)} + s^{(k)}}{5}$$
$$r^{(k+1)} = \frac{6 + p^{(k+1)} + q^{(k+1)} + s^{(k)} + t^{(k)}}{4}$$
$$s^{(k+1)} = \frac{2 - r^{(k+1)} + 2t^{(k)}}{4}$$
$$t^{(k+1)} = \frac{-1 - q^{(k+1)} + r^{(k+1)} - s^{(k+1)}}{4}$$

Kriteria berhenti:

$$\varepsilon_k = \max_i \left| x_i^{(k)} - x_i^{(k-1)} \right| < 0.001$$

Jacobi, iterasi 1 ($x^0 = 0$):

$$p^{(1)} = \frac{-1+0+0}{5} = -0.2$$
$$q^{(1)} = \frac{2+0+0+0}{5} = 0.4$$
$$r^{(1)} = \frac{6+0+0+0+0}{4} = 1.5$$
$$s^{(1)} = \frac{2-0+2(0)}{4} = 0.5$$
$$t^{(1)} = \frac{-1-0+0-0}{4} = -0.25$$
$$x^{(1)} = (-0.2, 0.4, 1.5, 0.5, -0.25), \varepsilon_1 = 1.5$$

Jacobi, iterasi 2:

$$p^{(2)} = \frac{-1+0.4+1.5}{5} = \frac{0.9}{5} = 0.18$$
$$q^{(2)} = \frac{2+(-0.2)+1.5+0.5}{5} = \frac{3.8}{5} = 0.76$$
$$r^{(2)} = \frac{6+(-0.2)+0.4+0.5+(-0.25)}{4} = \frac{6.45}{4} = 1.6125$$
$$s^{(2)} = \frac{2-1.5+2(-0.25)}{4} = \frac{0}{4} = 0$$
$$t^{(2)} = \frac{-1-0.4+1.5-0.5}{4} = -\frac{0.4}{4} = -0.1$$
$$x^{(2)} = (0.18, 0.76, 1.6125, 0, -0.1), \varepsilon_2 = 0.5$$

Gauss-Seidel, iterasi 1:

$$p^{(1)} = \frac{-1+0+0}{5} = -0.2$$
$$q^{(1)} = \frac{2+(-0.2)+0+0}{5} = \frac{1.8}{5} = 0.36$$
$$r^{(1)} = \frac{6+(-0.2)+0.36+0+0}{4} = \frac{6.16}{4} = 1.54$$
$$s^{(1)} = \frac{2-1.54+2(0)}{4} = \frac{0.46}{4} = 0.115$$
$$t^{(1)} = \frac{-1-0.36+1.54-0.115}{4} = \frac{0.065}{4} = 0.01625$$
$$x^{(1)} = (-0.2, 0.36, 1.54, 0.115, 0.01625), \varepsilon_1 = 1.54$$

Gauss-Seidel, iterasi 2:

$$p^{(2)} = \frac{-1 + 0.36 + 1.54}{5} = \frac{0.9}{5} = 0.18$$
$$q^{(2)} = \frac{2 + 0.18 + 1.54 + 0.115}{5} = \frac{3.835}{5} = 0.767$$
$$r^{(2)} = \frac{6 + 0.18 + 0.767 + 0.115 + 0.01625}{4} = \frac{7.07825}{4} = 1.7695625$$
$$s^{(2)} = \frac{2 - 1.7695625 + 2(0.01625)}{4} = \frac{0.2629375}{4} = 0.065734$$
$$t^{(2)} = \frac{-1 - 0.767 + 1.7695625 - 0.065734}{4} = -\frac{0.0631715}{4} = -0.015793$$
$$x^{(2)} = (0.18, 0.767, 1.7696, 0.065734, -0.015793), \varepsilon_2 = 0.407$$

Kelemahan dan kondisi khusus: kedua metode hanya terjamin konvergen karena A dominan diagonal (lemah) dan tidak dapat direduksi, seperti dibahas pada nomor 1. Hitungan manual rawan salah hitung karena tiap iterasi memuat 5 persamaan, dan satu kesalahan akan terbawa ke iterasi berikutnya.

Perbandingan hasil manual Anda dengan perhitungan ulang dan output program:

1. Jacobi iterasi 1 benar dan cocok dengan program. Pada iterasi 2, nilai p, q, r, s Anda benar, tetapi $t\hat{}(2) = 0.025$ tidak cocok. Hasil yang benar adalah t^(2) = −0.1 (program menampilkan −0.1000).
2. Gauss-Seidel iterasi 1: nilai $t\hat{}(1) = 0.045$ tidak cocok. Hasil yang benar adalah 0.01625 (program menampilkan 1.6250e-02). Karena t^(1) salah, r^(2) = 1.77675 dan s^(2) = 0.07831 Anda ikut bergeser. Nilai yang benar adalah r^(2) = 1.7696 dan s^(2) = 0.065734. Bahkan dengan $t^1 = 0.045$ sekalipun, t^(2) = 0.00244 tetap tidak cocok. Nilai itu seharusnya −0.01714, sehingga ada kesalahan hitung tambahan pada baris t.
3. Hasil akhir yang Anda tulis tidak cocok dengan program, dan nilai-nilainya lebih jauh dari solusi eksak.

Hasil program (nomor 1):

$$x_J = (0.32509187, 0.83204321, 1.79448861, 0.04155804, -0.01985528), k = 9$$
$$x_{GS} = (0.32568239, 0.83258862, 1.79505801, 0.04134867, -0.01971982), k = 5$$

Hasil manual Anda:

$$x_J = (0.326686, 0.834264, 1.799433, 0.045877, -0.008767), k = 9$$
$$x_{GS} = (0.327040, 0.834548, 1.799614, 0.045713, -0.008734), k = 6$$

Solusi eksak (dari Gauss):

$$x = (0.325444, 0.832347, 1.794872, 0.041420, -0.019724)$$

Contoh galat hasil manual Anda terhadap solusi eksak, pada variabel t dan r:

|−0.008767−(−0.019724)|=0.010957
|1.799433−1.794872|=0.004561

Galat ini jauh lebih besar dari galat program (maksimum 3.83×10^(−4)). Jadi nilai akhir Anda tidak mungkin berasal dari iterasi yang berhenti pada ε < 0.001. Dugaan penyebabnya: kesalahan hitung t pada iterasi awal terbawa terus sampai akhir, atau nilai akhir berasal dari proses lain, bukan dari output program ini. Jumlah iterasi Gauss-Seidel Anda (k = 6) juga berbeda dari program (k = 5).

Bukti bahwa t^(2) = −0.1 yang benar: iterasi 3 Jacobi dari x^(2) yang benar menghasilkan baris 3 tabel program.

$$s^{(3)} = \frac{2 - 1.6125 + 2(-0.1)}{4} = \frac{0.1875}{4} = 0.046875$$
$$t^{(3)} = \frac{-1 - 0.76 + 1.6125 - 0}{4} = -\frac{0.1475}{4} = -0.036875$$

Hasilnya cocok dengan program (s = 0.0469, t = −0.0369). Bila memakai t^(2) = 0.025, s^(3) akan menjadi 0.109375, yang tidak cocok dengan program.

Saran untuk laporan: perbaiki iterasi 1 dan 2 seperti di atas, lalu pakai hasil akhir dari program (nomor 1) sebagai hasil Jacobi dan Gauss-Seidel. Bila dosen meminta iterasi manual sampai konvergen, lanjutkan iterasi 3 dst. dari nilai yang sudah benar. Nilai tiap iterasi dapat dicek ke tabel iterasi pada output nomor 1.

Kesimpulan manual: Jacobi memerlukan 9 iterasi dan Gauss-Seidel 5 iterasi untuk mencapai ε < 0.001. Gauss-Seidel lebih cepat konvergen karena langsung memakai nilai terbaru.

Cara memakai: di Word, tekan Alt + = untuk membuka kotak persamaan, tempel isi satu kotak kode, lalu tekan Enter agar berubah menjadi rumus. Lakukan satu per satu per kotak kode.

Catatan simbol: saya belum menguji sintaks ini langsung di Word. Bagian yang berpotensi gagal terkonversi adalah superskrip bertingkat seperti p^((k+1)), tanda |...| pada rumus ε_k, dan simbol ε_k yang dipakai sebagai nama selisih iterasi. Bila ada yang tidak terkonversi dengan benar, kabari saya dan akan saya sesuaikan penulisannya.

BAB III

ANALISIS PRAKTIKUM

1. Kesimpulan dari Perbandingan Analisis Hasil Source Code dan Analisis Numerik

Kesimpulan ini memakai nilai manual yang sudah dikoreksi pada nomor 4 (Jacobi t^(2) = −0.1, Gauss-Seidel t^(1) = 0.01625). Jika laporan Anda masih memuat nilai awal (t^(2) = 0.025, t^(1) = 0.045, dan hasil akhir 0.326686 dst.), pernyataan "manual sama dengan program" tidak berlaku untuk nilai-nilai itu. Perbaiki dulu angkanya sebelum menyalin kesimpulan ini.

Analisis numerik manual dari Anda hanya mencakup Jacobi dan Gauss-Seidel. Untuk Gauss, Gauss-Jordan, dan LU, pembandingnya adalah verifikasi manual saya (substitusi balik, L·U = A, determinan) pada nomor 1 dan 2.

i. Perbandingan per metode

Metode Jacobi

Yang sama:

- Rumus iterasi, tebakan awal, dan kriteria berhenti identik.
- Iterasi 1 identik:

$$x^{(1)} = (-0.2, 0.4, 1.5, 0.5, -0.25), \varepsilon_1 = 1.5$$

- Iterasi 2 identik setelah koreksi:

$$x^{(2)} = (0.18, 0.76, 1.6125, 0, -0.1), \varepsilon_2 = 0.5$$

Yang berbeda:

- Program memakai perulangan dengan indeks idx dan menghitung ε_k otomatis sampai ε_k < 0.001, yaitu pada iterasi ke-9.
- Manual menuliskan tiap persamaan secara eksplisit dan hanya dua iterasi, sebagai contoh langkah.

Yang dibuktikan: algoritma jacobi di program adalah implementasi langsung dari rumus manual. Selisih terakhir pada iterasi ke-9 adalah 0.000597 < 0.001.

Metode Gauss-Seidel

Yang sama:

- Rumus dan tebakan awal sama, dengan nilai terbaru langsung dipakai.

- Iterasi 1 dan 2 identik setelah koreksi:

$$x^{(1)} = (-0.2, 0.36, 1.54, 0.115, 0.01625), \varepsilon_1 = 1.54$$
$$x^{(2)} = (0.18, 0.767, 1.7696, 0.065734, -0.015793), \varepsilon_2 = 0.407$$

Yang berbeda:

- Di program, pembaruan nilai terbaru terjadi otomatis karena x(i) ditimpa di tempat. Di manual, hal itu harus ditulis eksplisit lewat superskrip (k+1) pada ruas kanan.
- Program berhenti pada iterasi ke-5 dengan selisih 0.000689. Hasil manual Anda menyebut k = 6, yang tidak sesuai.

Yang dibuktikan: pembaruan nilai terbaru membuat Gauss-Seidel konvergen lebih cepat (5 iterasi berbanding 9 pada Jacobi).

Metode langsung (Gauss, Gauss-Jordan, LU)

Yang sama: ketiganya menghasilkan solusi identik. Hasil ini cocok dengan verifikasi manual (substitusi balik ke kelima persamaan memberi ruas kanan b persis).

$$p = \frac{165}{507} = 0.32544379$$
$$q = \frac{422}{507} = 0.83234714$$
$$r = \frac{910}{507} = 1.79487179$$
$$s = \frac{21}{507} = 0.04142012$$
$$t = -\frac{10}{507} = -0.01972387$$

Yang berbeda:

- Gauss hanya mengeliminasi di bawah pivot, lalu memakai substitusi mundur.
- Gauss-Jordan mengeliminasi di atas dan di bawah pivot sehingga tidak perlu substitusi mundur.
- LU memisahkan faktorisasi dari penyelesaian, yaitu Ly = b lalu Ux = y.

Yang dibuktikan: determinan 1521 ≠ 0 menjamin solusi tunggal, dan L·U = A terbukti pada pengecekan baris 5.

Analisis galat (nomor 2)

Hitungan manual galat Jacobi dan Gauss-Seidel sama dengan output program:

$$E_{maks}^{Jacobi} = 3.8318 \times 10^{-4}$$
$$E_{maks}^{GS} = 2.4148 \times 10^{-4}$$

Galat metode langsung hanya galat pembulatan mesin ($sekitar\ 10^{-16}$), sedangkan galat metode iteratif adalah galat pemotongan akibat iterasi dihentikan. Kedua galat iteratif lebih kecil dari toleransi 0.001.

Running time (nomor 3)

Tidak ada pembanding manual untuk waktu eksekusi. Yang bisa dibandingkan adalah teori dan pengukuran:

- Teori: metode langsung $O(n^3)$, metode iteratif $O(k \cdot n^2)$.
- Pengukuran: pada n = 5, Gauss tercepat dan Jacobi terlambat, karena overhead interpreter Octave lebih menentukan daripada jumlah operasi.
- Rasio waktu Jacobi terhadap Gauss-Seidel (1.78) mendekati rasio iterasinya (9/5 = 1.8), sehingga biaya per iterasi kedua metode hampir sama.

ii. Kesimpulan umum

Kesamaan hasil antar metode:

- Kelima metode menuju solusi yang sama, yaitu (p, q, r, s, t) = (0.325444, 0.832347, 1.794872, 0.041420, −0.019724).
- Metode langsung identik sampai sekitar $10\text{^}(-16)$. Metode iteratif berbeda sekitar 10^(−4) dari solusi eksak, sesuai toleransi.
- Analisis source code dan analisis manual saling menguatkan: setiap iterasi manual (setelah koreksi) sama dengan baris tabel iterasi program.

Kelebihan dan kekurangan tiap metode:

- Gauss: paling sedikit operasi di antara metode langsung dan tercepat pada pengujian. Kekurangannya, gagal bila matriks singular dan harus diulang penuh untuk setiap vektor b baru.
- Gauss-Jordan: langsung menghasilkan x tanpa substitusi mundur dan mudah dikembangkan untuk invers matriks. Kekurangannya, operasinya sekitar 1.5 kali Gauss.
- LU: faktorisasi dapat dipakai ulang untuk banyak vektor b. Kekurangannya, versi Doolittle pada kode ini tanpa pivoting, sehingga pembagian nol terjadi bila u_ii = 0.
- Jacobi: sederhana dan mudah diparalelkan karena tiap komponen independen. Kekurangannya, konvergen paling lambat (9 iterasi) dan galatnya paling besar.
- Gauss-Seidel: lebih cepat dan lebih akurat daripada Jacobi pada toleransi yang sama. Kekurangannya, urutan komputasi sekuensial dan hasilnya bergantung pada urutan persamaan.

Kapan metode tertentu lebih cocok:

- SPL kecil dan padat (seperti 5×5 ini): Gauss, karena eksak dan paling cepat.
- Banyak SPL dengan matriks A yang sama dan b berbeda: LU.
- Matriks besar, jarang (sparse), dan dominan diagonal: Gauss-Seidel, karena biaya $O(k \cdot n^2)$ dan tidak memerlukan penyimpanan fill-in.
- Bila diperlukan komputasi paralel: Jacobi.

Catatan stabilitas numerik:

- Pivoting parsial pada Gauss dan Gauss-Jordan mencegah pembagian nol dan menekan galat pembulatan. Untuk matriks ini, tidak ada pertukaran baris yang dibutuhkan.
- Syarat konvergensi iteratif terpenuhi karena A dominan diagonal lemah dan tidak dapat direduksi:

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$$

- Kriteria berhenti memakai selisih antar iterasi ε_k, bukan galat sebenarnya. Pada kasus ini ε_k lebih besar dari galat sebenarnya, sehingga bersifat konservatif:

$$E_{maks}^{Jacobi} = 3.83 \times 10^{-4} < \varepsilon_9 = 5.97 \times 10^{-4}$$
$$E_{maks}^{GS} = 2.41 \times 10^{-4} < \varepsilon_5 = 6.89 \times 10^{-4}$$

- Hasil waktu eksekusi bergantung pada komputer dan interpreter, sehingga yang dipakai sebagai kesimpulan adalah urutan relatifnya.

Kesimpulan akhir: untuk SPL 5×5 ini, metode langsung unggul dalam akurasi dan kecepatan, sedangkan metode iteratif (terutama Gauss-Seidel) layak dipakai bila toleransi 0.001 sudah memadai dan sistemnya besar dan jarang. Kesesuaian iterasi manual (setelah koreksi) dengan tabel iterasi program membuktikan bahwa implementasi Octave benar.